

command. By issuing the above command, we not only created a pool but also implicitly created a dataset (more specifically, a filesystem) too, which will be mounted by default at location `/K7`. It is important to avoid any pool name whose name clashes with directories under `/` [root directory]. However, if you want to explicitly specify the mount point, say at `/mnt/k7` or elsewhere, then execute the following:

```
zpool create -m /mnt/k7 K7 sda10
```

...or if pool `'K7'` already exists:

```
zfs set mountpoint=/mnt/k7 K7
```

However, after this, `K7` won't be mounted anywhere. So we need to issue either auto mount on all filesystems by issuing the following command:

```
zfs mount -a
```

...or any specific filesystem as:

```
zfs mount K7
```

For unmounting we use the *unmount* option instead of *mount* in the above commands.

Also, at any point in time, if you want to list all the pools in your system, execute the command given below:

```
zpool list
```

The health status of the pool can be checked with the following:

```
zpool status
```

This command can take the optional arguments `-x` and `-v` for a quick overview and verbose status, respectively.

Since we have created a pool named `'K7'` along with a filesystem with the same name and mounted it at `/mnt/k7`, to properly harvest the pool we may need more filesystems suitably named in the pool `'K7'`. This can be achieved by using the dataset specific command *zfs* rather than the pool command *zpool*.

For example:

```
zfs create K7/ajc
```

...will create a filesystem mounted at a sub-directory `'ajc'` in a directory where `K7` is mounted, which in our case will be `/mnt/k7/ajc`. Similar to specifying mounting options for pools as mentioned above, filesystems also have options like:

```
zfs create -o mountpoint=/mnt/k7/ajc K7/ajc
```

Or if you want to change the mount point of an already created filesystem, use:

```
zfs set mountpoint=/mnt/k7/ajc K7/ajc
```

It is quite possible that after some time the space you allocated for the pool may run out. Using the in-built compression can be a temporary, yet ready-made solution for such a situation.

```
zfs set compression=on K7/ajc
```

Another way to tackle this is to add devices to the pool with the required device space, which will be added to the space already available.

```
zpool add K7 sda11
```

Also, as a counter operation to *add*, we also have *remove* to remove any added devices from the pool but with the restriction that removal can be performed only on hot spare (that is, inactive devices made active when the system is degraded) devices.

Like *mountpoint* and *compression*, many other properties of a filesystem like `'quota'`, `'reservation'`, etc, can also be set as:

```
zfs set quota=3G K7/ajc
```

```
zfs set reservation=1G K7/ajc
```

Properties of a filesystem can be viewed using *get* as follows:

```
zfs get quota K7/ajc
```

And to see all properties, issue the following command:

```
zfs get all K7/ajc
```

As mentioned earlier, ZFS gives a lot of importance to data validation, which is also called scrubbing, and this can be performed on any of the filesystems using the command *scrub*:

```
zpool scrub K7
```

If at any point you want to see all the commands you issued on pools, use:

```
zpool history
```

Or for a particular pool like `K7`, issue the following:

```
zpool history K7
```

Likewise, use *iostat* to get a count of I/O